

Technical Design Document

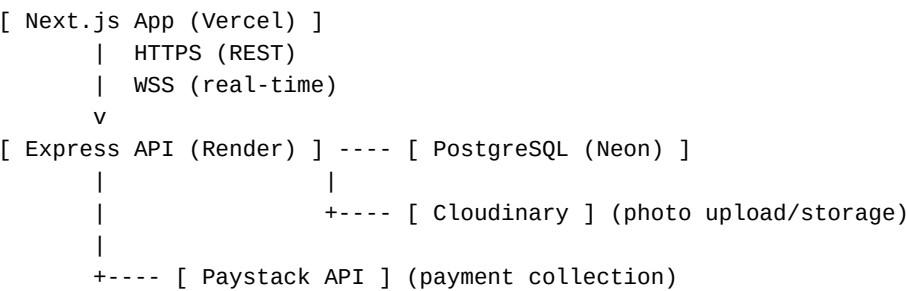
Campus Errand Marketplace — OAU Student Task Platform

1. Tech Stack Summary

Layer	Choice	Notes
Frontend	Next.js (App Router)	Route groups split marketing pages from the authenticated app
Backend	Node.js / Express	Standalone API, deployed separately from the frontend
Database	PostgreSQL (Neon)	Permanent free tier, no expiry
Real-time	WebSockets (Socket.io)	Task status updates, claim notifications
Auth	Custom (Express + bcrypt + JWT)	Includes matric number / school email verification logic
Photo storage	Cloudinary	Proof-of-completion photos, auto-compression
Payments	Paystack (Starter Business)	Manual release flow, see Section 5
Frontend hosting	Vercel	Free tier, free subdomain for now
Backend hosting	Render	Free tier; free web service spins down after 15 min idle

2. System Architecture

The system is split into two independently deployed services that communicate over HTTPS/WSS:



The frontend never talks to Postgres, Cloudinary, or Paystack directly — all of it is proxied through the Express API, which is the only service holding credentials for those systems. This keeps secrets off the client and gives a single place to enforce business rules (e.g. a payout can only be released through a server-side check that the task is in Confirmed status).

3. Frontend / Backend Responsibility Split

3.1 Frontend (Next.js on Vercel)

- Public marketing routes (landing page, about) — statically generated, no auth required.
- Authenticated app routes (dashboard, task browsing, task detail, profile, ratings) — behind auth, PWA behavior (installability, manifest, service worker) applies to this route group only.
- All data fetching goes through the Express API; no direct database or third-party service calls from the frontend.
- Maintains the WebSocket client connection for live task status updates while in the app shell.

3.2 Backend (Express on Render)

- REST API for all CRUD operations: users, tasks, claims, ratings, disputes.
- WebSocket server for broadcasting task status changes to relevant connected clients.
- Auth logic: signup/login, password hashing, JWT issuance and verification, matric number / school email domain validation.
- Paystack integration: initializing payment collection, verifying webhooks, and triggering manual payout release.
- Cloudinary integration: generating signed upload URLs for proof-of-completion photos.
- Enforces all business rules server-side (task state transitions, deadline checks, payout eligibility) so the frontend cannot bypass them.

4. Database Schema (PostgreSQL)

This maps directly to the task state machine defined in the PRD (Section 5.3).

4.1 users

id	UUID PRIMARY KEY
matric_number	VARCHAR UNIQUE NOT NULL
school_email	VARCHAR UNIQUE
password_hash	VARCHAR NOT NULL
full_name	VARCHAR NOT NULL
phone_number	VARCHAR
avg_rating	DECIMAL DEFAULT NULL
trust_tier	VARCHAR DEFAULT 'new' -- new established trusted
created_at	TIMESTAMP DEFAULT now()

4.2 tasks

id	UUID PRIMARY KEY
poster_id	UUID REFERENCES users(id)
doer_id	UUID REFERENCES users(id) NULL
title	VARCHAR NOT NULL
description	TEXT NOT NULL
proof_requirement	TEXT NOT NULL -- what counts as valid completion proof
task_price	DECIMAL NOT NULL
transport_estimate	DECIMAL NOT NULL
total_price	DECIMAL NOT NULL

location_lat	DECIMAL	
location_lng	DECIMAL	
deadline_at	TIMESTAMP NULL	-- hard deadline, optional
claim_expires_at	TIMESTAMP NULL	-- unclaimed timeout
status	VARCHAR NOT NULL	-- open claimed in_progress completed -- confirmed expired missed_deadline disputed
created_at	TIMESTAMP DEFAULT now()	
claimed_at	TIMESTAMP NULL	
completed_at	TIMESTAMP NULL	
confirmed_at	TIMESTAMP NULL	

4.3 task_proofs

id	UUID PRIMARY KEY
task_id	UUID REFERENCES tasks(id)
photo_url	VARCHAR NOT NULL -- Cloudinary URL
uploaded_at	TIMESTAMP DEFAULT now()

4.4 transactions

id	UUID PRIMARY KEY
task_id	UUID REFERENCES tasks(id)
poster_id	UUID REFERENCES users(id)
doer_id	UUID REFERENCES users(id)
amount	DECIMAL NOT NULL
paystack_ref	VARCHAR NOT NULL
status	VARCHAR NOT NULL -- collected released refunded
collected_at	TIMESTAMP NULL
released_at	TIMESTAMP NULL

4.5 ratings

id	UUID PRIMARY KEY
task_id	UUID REFERENCES tasks(id)
rater_id	UUID REFERENCES users(id)
ratee_id	UUID REFERENCES users(id)
score	INTEGER NOT NULL -- 1-5
comment	TEXT NULL
created_at	TIMESTAMP DEFAULT now()

4.6 disputes

id	UUID PRIMARY KEY
task_id	UUID REFERENCES tasks(id)
raised_by	UUID REFERENCES users(id)
reason	TEXT NOT NULL
status	VARCHAR NOT NULL -- open resolved
resolution	TEXT NULL
created_at	TIMESTAMP DEFAULT now()
resolved_at	TIMESTAMP NULL

5. Auth Flow

- **Signup:** user submits matric number, school email, full name, and password. Backend validates the school email domain, checks matric number uniqueness, hashes the password with bcrypt, and creates the user record.
- **Login:** backend verifies the password hash and issues a signed JWT (short-lived access token). A refresh token strategy should be used so users are not forced to re-login frequently.
- **Session handling:** JWT is sent as an httpOnly cookie (not accessible to client-side JavaScript) to reduce exposure to token theft via cross-site scripting.
- **Protected routes:** Express middleware verifies the JWT on every request to a protected endpoint before allowing access.
- **Security essentials:** bcrypt with an adequate salt round count, rate-limiting on login attempts to prevent brute-force attacks, and token expiry enforced server-side.

6. Payment Flow

- Poster initiates payment for a task — backend calls Paystack to collect the task total upfront.
- Paystack webhook confirms successful collection; backend updates the transaction status to “collected” and the task status to “claimed” once a doer accepts it.
- On task confirmation, payout to the doer is triggered manually (Starter Business tier has no Transfers API access) — an admin action in a lightweight internal tool marks the transaction as “released” once the transfer is sent from the platform's own bank account.
- On dispute or missed deadline, the transaction is marked “refunded” and the poster is refunded via Paystack's refund API.

7. API Endpoint Overview

Endpoint	Method	Purpose
/auth/signup	POST	Create account, verify matric number/email
/auth/login	POST	Authenticate, issue JWT
/tasks	GET	List open tasks, proximity-sorted
/tasks	POST	Create a task
/tasks/:id/claim	POST	Claim an open task (first-come)
/tasks/:id/complete	POST	Mark completed, requires proof photo
/tasks/:id/confirm	POST	Poster confirms, triggers payout
/tasks/:id/dispute	POST	Flag an issue, freezes payout

/ratings	POST	Submit a rating after task completion
/users/:id	GET	Public profile, rating history

8. WebSocket Events

- **task:claimed** — sent to the poster when their task is claimed.
- **task:status_changed** — sent to both parties on any status transition.
- **task:new_nearby** — optionally sent to nearby users when a new task is posted in their zone.

9. Known Constraints & Risks

- Render's free web service spins down after 15 minutes of inactivity; WebSocket connections drop and reconnect with a 30-60 second cold start after idle periods. Acceptable at current scale.
- Manual payout release is a bottleneck as usage grows; automating this requires formal business registration to unlock Paystack's Transfers API.
- Self-built auth carries more security responsibility than a managed provider; the security essentials listed in Section 5 are non-negotiable minimums, not optional hardening.